

代码片段 6.12 PCB 结构 (第五版): 加入进程执行状态, 去掉 `is_exit`

```
1 enum exec_status {NEW, READY, RUNNING, ZOMBIE, TERMINATED};
2
3
4 struct process_v5 {
5     // 处理器上下文
6     struct context *ctx;
7     // 虚拟地址空间 (包含页表基地址)
8     struct vmSPACE *vmSPACE;
9     // 内核栈
10    void *stack;
11    // 进程标识符
12    int pid;
13    // 退出状态
14    int exit_status;
15    // 子进程列表
16    pcb_list *children;
17    // 执行状态
18    enum exec_status exec_status;
19 };
```

代码片段 6.13 基于进程执行状态的调度选择的伪代码实现

```
1 struct process* pick_next(void)
2 {
3     // 遍历进程列表, 寻找下一个可调度 (处于就绪状态) 的进程
4     for (struct process *proc : all_processes) {
5         if (proc->exec_status == READY) {
6             // 将上一个正在运行的进程变为就绪, 然后返回
7             curr_proc->exec_status = READY;
8             return proc;
9         }
10    }
11 }
12 void schedule()
13 {
14     curr_proc = pick_next();
15     // 将选中的进程执行状态变为运行
16     curr_proc->exec_status = RUNNING;
17 }
```

图 6.3 仅展示了比较基本的五种执行状态, 为了方便内核进行进程管理, 还可以引入更多的状态类型。例如在进程睡眠的例子中, 如果设定的时间过长, 调用 `process_sleep` 的进程可能会被内核调度很多次, 但都因为时间未到而再次调用 `schedule`, 白白浪费 CPU 资源。为了解决这一问题, 内核可以引入阻塞 (Blocked) 状态, 对应需要在内核中等待、无法马上回到用户态执行的进程。同样是进程睡眠的例子, 内核可以将调用 `process_sleep` 的进程执行状态改为阻塞状态, 从而避免其被调度, 节约 CPU 资

源。而当进程睡眠的时间超过参数指定的长度后，内核会将进程“唤醒”，即将其执行状态改回就绪状态。阻塞和唤醒机制的内核支持将在第9章详细介绍。

练习

- 6.2 在6.1.4节 grep 与 shell 的例子中，grep 进程在其生命周期中的执行状态是如何变化的？其状态变化是否存在多种可能？

6.2 案例分析：ChCore 微内核的进程管理

本节主要知识点

- 什么是能力组？其在 ChCore 微内核中有什么作用？
- 微内核的进程管理接口如何实现？
- 微内核的进程管理与宏内核存在哪些不同？

前面的章节主要从宏内核视角出发，介绍了进程相关数据结构及管理接口的实现。在宏内核中，与进程相关的数据结构（如 PCB）均放在内核中，管理接口也以系统调用的形式暴露给用户，核心功能全部在内核中完成。但在微内核中，包括进程管理在内的操作系统功能被拆分并移入用户态，因此与宏内核存在较大不同。本节将以 ChCore 为例，分析微内核的进程管理机制是如何实现的。

6.2.1 进程管理器与分离式 PCB

ChCore 采用微内核设计，它将操作系统的功能进行解耦，并将很多功能以模块形式移到用户态。进程管理也不例外，ChCore 将这部分功能移到用户态，其对应模块称为进程管理器（Process Manager）。当用户调用与进程管理相关的调用（如进程创建和终止）时，实际上是调用进程管理器，并由进程管理器与内核交互，最终达到管理进程的目的。这也使得 PCB 的结构由宏内核的集中式向分离式方向发展。图 6.4 展示了由进程管理器和内核共同管理的分离式 PCB 结构。从图中可以看出，进程的 PCB 结构被分割成两部分，分别放在内核态和用户态。

PCB 内核态部分：cap_group

ChCore PCB 在内核态的部分称为 cap_group，是能力组（Capability group）的简称。由于程序运行过程中需要不同类型的资源（如 CPU 和内存等），为了便于对资源进行管理，ChCore 对内核资源进行了抽象，每种资源对应一种类型的对象（Object），而访问某个具体对象所需的“凭证”就是能力（Capability）。由于 ChCore 将进程作为资源

分配和管理的基本单位，因此进程自然也就成为拥有若干能力的“能力组”——cap_group。

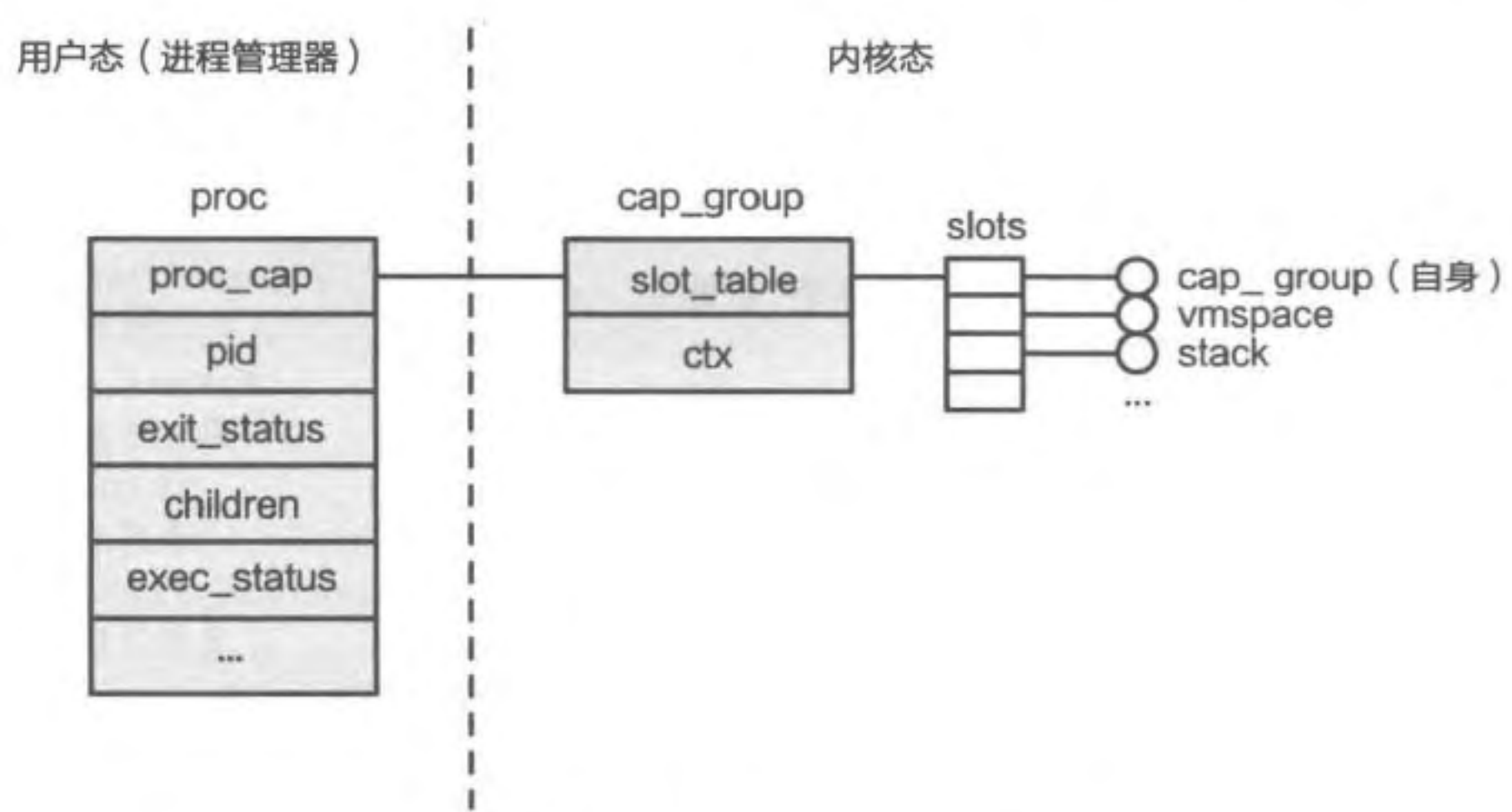


图 6.4 ChCore 微内核的分离式 PCB 结构

ChCore 的 cap_group 只包含两部分：存放对象的 slot_table 和处理器上下文 ctx。其中，slot_table 包含一个对象数组 slots，维护进程所持有的全部对象，包括它自身（进程本身也是对象）、虚拟地址空间、用户栈对应的物理内存等。而这些对象在数组中的偏移量就是它们对应的能力。另外，ChCore PCB 中的处理器上下文与宏内核 PCB 中保存的处理器上下文结构相同。

读者可能会好奇，处理器上下文和虚拟地址空间都有了，为什么 ChCore 的 PCB 中没有包含内核栈？实际上，ChCore 依然为每个进程准备了内核栈，虽然它没有保存在 PCB 中，但仍可以通过间接索引的方式找到其地址，我们将在进程切换部分展开介绍。

PCB 用户态部分：proc

通过观察 cap_group 结构不难发现，其结构与 6.1 节介绍的 PCB 第一版的结构很相似，这是因为处理器上下文和虚拟地址空间都是内核提供进程支持的关键，难以在用户态高效地实现。而前面介绍的进程创建、退出、等待等功能所需的其他信息则都移动到了 PCB 的用户态部分——proc 结构体里。proc 结构体中还保存了与 cap_group 对应的能力 proc_cap，便于进程操作的实现。

6.2.2 ChCore 的进程操作：以进程创建为例

ChCore 使用的进程创建接口为 spawn，该接口与 process_create 功能类似，都是从头开始创建进程。代码片段 6.14 展示了 ChCore 中 spawn 的伪代码实现，其步骤依次为：可执行文件加载、创建进程、分配用户栈、初始化用户栈、映射虚拟内存。由于这些步骤与 process_create 相似，本节不再赘述。但需要注意的是，由于进程管

理器已经移到了用户态，因此 `spawn` 并不是系统调用，而是处于用户态的系统服务提供的接口。在 `spawn` 执行过程中，进程管理器会使用必要的系统调用与内核交互，最终完成进程创建的功能。

代码片段 6.14 ChCore 中进程创建 (`spawn`) 的伪代码实现

```

1 int spawn(char *path, ...)
2 {
3     // 根据指定的文件路径，加载需要执行的文件（同时创建相关物理内存对象 PMO）
4     struct user_elf *elf = readelf(path);
5
6     // 进程管理器获取一个新的 pid 作为进程标识符
7     int pid = alloc_pid();
8     // 创建进程（包括创建 PCB、虚拟地址空间、处理器上下文和内核栈）
9     int new_process_cap = create_cap_group(pid, path, ...);
10
11     // 为用户栈创建物理内存对象 PMO
12     int stack_cap = create_pmo(...);
13     // 利用数组构造初始执行环境
14     char init_env[ENV_SIZE];
15     construct_init_env(init_env, elf, ...);
16     // 更新执行环境到栈对应的 PMO 中
17     write_pmo(stack_cap, init_env, ...);
18
19     // 构建请求，用于映射栈对应的 PMO
20     struct pmo_map_request requests[MAX_REQ_SIZE];
21     add_request(requests, stack_cap, STACK_VADDR, ...);
22     // 构建请求，用于映射可执行文件中需要加载的段对应的 PMO
23     for (struct user_elf_seg seg: elf->loadable_segs)
24         add_request(requests, seg.pmo, seg.p_vaddr, ...);
25
26     // 完成上述 PMO 的实际映射
27     map_pmos(new_process_cap, requests, ...);
28     // 完成其他部分的初始化（包括 proc 结构体）并返回
29     ...
30 }

```

那么，`spawn` 的执行过程中到底有哪些部分是在内核中完成的呢？图 6.5 展现了 `spawn` 函数的控制流在用户态和内核态之间切换的全过程。注意，虽然负责解析 ELF 文件的 `readelf` 函数主要在用户态执行，但它需要 `create_pmo`、`map_pmo` 等系统调用来创建新的物理内存对象，用于保存之后需要映射的程序节。从图中不难看出，当需要分配更多内核资源，或是需要对内核资源进行修改时，就要使用系统调用进入内核，其他功能都可以在用户态完成。如果要实现其他功能，也可以使用相似的方法。例如对于进程退出接口 `exit`，只需要使用相应的系统调用释放创建的对象，而与用户态 PCB 相关的部分（如进程标识符）就直接在用户态释放即可。

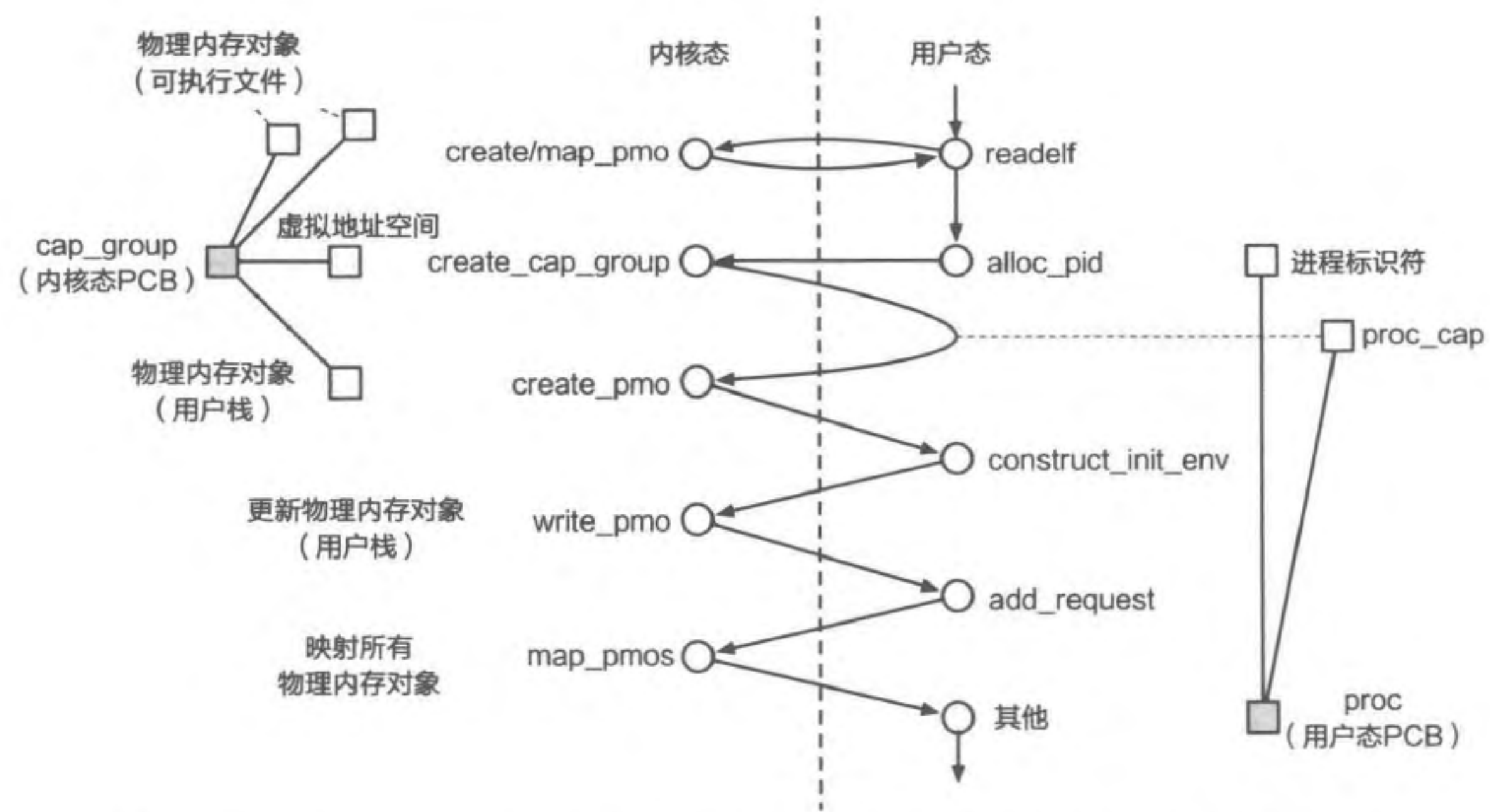


图 6.5 ChCore 的 spawn 函数控制流（圆圈表示执行的函数，方块表示新创建的数据结构）

6.3 案例分析：Linux 的进程创建

本节主要知识点

- ❑ Linux 中最为经典的进程创建接口 fork 是如何实现的？
- ❑ fork 有哪些优点和缺点？
- ❑ Linux 中还有哪些创建进程的方法？它们都有什么特点？

本章已经从必要的功能出发，介绍了进程创建、退出、等待、睡眠相关系统调用的实现方法。而在现代操作系统中，为了满足应用的多样化需求，系统调用往往也是多样化的。对于同一个功能，操作系统可能会提供多种系统调用，它们具有不同的特点，并在不同的历史阶段和应用场景中扮演着重要角色。本节将以进程创建这一功能为例，介绍常见的系统调用及其发展历史。

6.3.1 经典的进程创建方法：fork

fork 是最为经典的进程创建方法，其使用历史非常悠久。早在 20 世纪 60 年代，由加州大学伯克利分校主导的分时系统研究项目 Genie 首先使用 fork 作为系统调用^[1]创建进程，而 UNIX 也沿用了这个设计^[2]。直至今日，fork 仍然广泛地应用于 shell、Web 服务器、浏览器等场景中。